

Système distribué = graphe

Sommets : site (entités de calcul) → peuvent exécuter du code
→ ils ont une mémoire.

arêtes : liens de communication

les sites peuvent envoyer des message aux autres sites

Difficulté : on veut résoudre un problème globale (il peut concerne tout le réseau : quelle est la taille du réseau $-|V|$)

les nœuds ont un "vision" du réseau et de la mémoire qui est locale (leurs voisinages)

la communication : se fait uniquement entre nœuds voisins
- envoi / réception de messages.

définition : algo distribué :

Algo 1

II VARIABLE

I CODE

I : le code va dépendre "d'événement extérieur" qui vont déclencher l'exécution du code.

- Événement extérieur 1 : Événement Initial : au dessus il existe au moins 1 nœud initial. Déclenché uniquement une fois. et sur des nœuds qui n'ayant pas déjà exécuté du code.

[for if while
:]

- Événement extérieur 2 : Réception message

régle ↗ : code informatique classique
gardées : + code de communication :
- envoyer message à un voisin.

Remarque : l'algo est dupliqué sur tous les nœuds du voisin.

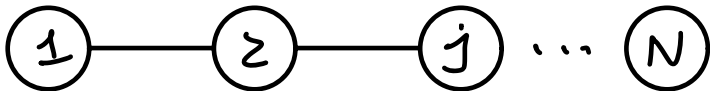
Variable : locales au nœud.

→ de connaissance (lecture)

- décrivent la topologie / l'ensemble des voisins.
- variables locales de calculs
 - (lecture et écriture)
- variables de communication (message)
 - décrit les variables contenue dans les messages.

Problème :

réseau en chaîne



$var_i = \alpha \in \mathbb{N}$

écrire un algorithme tq à la fin tous les noeuds connaissent la valeur maximale dans le réseau

Algo(i)

variables connaissances :

- $Vois_i$: les noeuds voisins du noeud i .
- val_i : valeur stockée initialement sur le noeud i

valeur locales

- max_i : initialisée à val_i max_i : initialisée à val_i

variables des communication

- $Aller(v)$ $Msg(v)$
- $Retour(v)$

Code :

Événement initial : sur 1 noeud de degré 1 "tous les noeuds" C

- $Aller(max_i)$ à $\{Vois_i\}$ Envoie max_i à $Vois_i$

Réception $Aller(m)$ Msg (variables de message) de j (noeud voisin).

- Si $m > max_i$
 - $max_i \leftarrow m$
- Si $|Vois_i| > 1$
 - Envoyer $Msg(max_i)$ à $\{Vois_i\} \setminus \{j\}$
- Sinon
 - Envoyer $Retour(max_i)$ à j

Réception de Retour(v) de j

$\max_i \leftarrow v$
Si $|Vois_i| > 1$
[Envoyer Retour($\max_i$) à $vois_i \setminus j$]

1^{er} version : les nœuds "savent" qu'il s'agit d'une chaîne.

2^{ème} version : ils ne connaissent pas la structure de graphe.

Complexité :

1. configuration et exécution
2. Complexité

ensemble des valeurs de toutes les variables de chaque site.

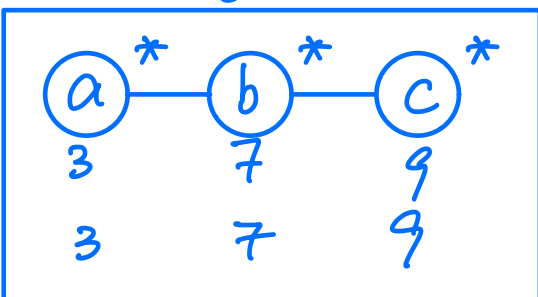
1. Configuration : état local de tous les sites + l'ensemble des messages en transit = état global du système

Une exécution d'un système distribué :

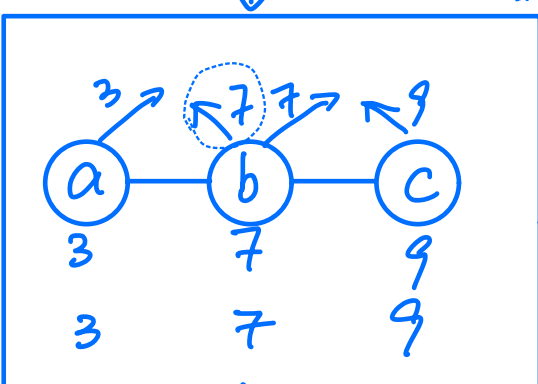
Def : site activable : site pour lequel il existe un événement extérieur activé et qui doit être traité.

Exécution : Une séquence de configuration $C_0, C_1, \dots, C_k, \dots$ tq entre deux configurations consécutives, tous les sites activables ont traité leurs événements extérieurs.

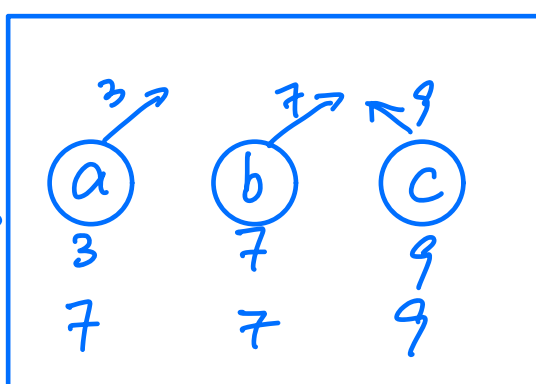
e_0 : configuration initiale.



e_1



↑
configuration



où a est activable.



plusieurs configurations possibles en fonction de quel message va être réceptionné par qui et va activer l'événement extérieur "réception de message".

Complexité en temps :

- synchrone : tous les messages transitent en 1 unité de temps
↳ complexité = longueur de la chaîne d'exécution.
- asynchrone : le temps de transit des message est arbitraire : arbre de l'exécution



complexité = hauteur de l'arbre des exécutions

Terminaison

- un algo termine si il existe 1 configuration dans laquelle il n'existe plus de messages en transit et aucun événement extérieur n'est en attente de traitement.
- explicite : si au moins 1 noeud "sait" que l'algorithme a terminé,

↳ on l'explícite avec "stop-global"

- implicite : l'algo termine mais aucun noeud ne peut savoir que c'est le cas.
- "stop-local" : noeud sont qu'il a traité sans forcément que l'algo est terminée.

Algorithm 2 Calcul du maximum dans une chaîne

——— **Connaissance** ———

$vois_i \subset V$: ensemble des sites voisins de i .

$valeur_i$: valeur du site.

——— **Variables** ———

max_i : maximum courant calculé jusqu'alors. Initialisée à $valeur_i$.

——— **Messages** ———

$Msg(v)$ avec v une valeur parmi l'ensemble des valeurs dont on veut calculer le maximum.

——— **Algorithme** ———

Initialement → /* Sur tous les sites */

Envoyer $Msg(valeur_i)$ à $vois_i$

Sur réception de $Msg(v)$ de j →

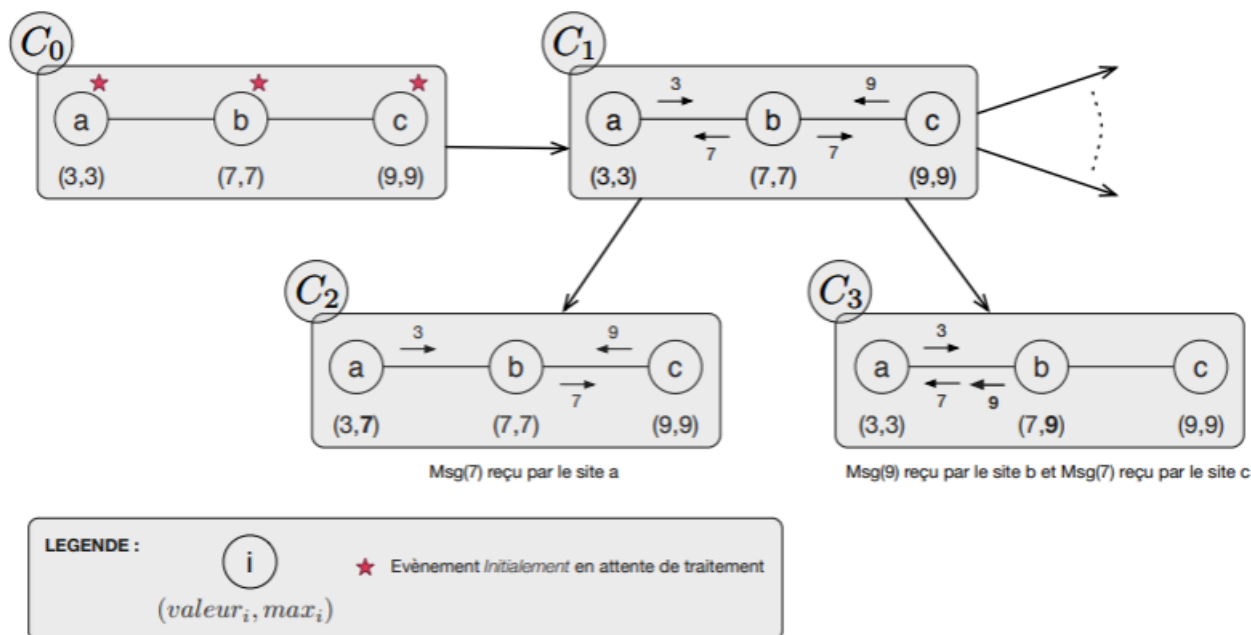
if $v > max_i$ then

$max_i \leftarrow v$

if $|vois_i| == 2$ then /* Un site au milieu */

Envoyer $Msg(v)$ à $vois_i \setminus \{j\}$

Préfixes d'exécutions possibles de l'algorithme :



2 Exécution et Complexité

2.1 Quelques définitions

Configuration

- État local (d'un site) : ensemble des valeurs de toutes les variables du site.
- Messages en transit : les messages dans les canaux de communications (messages dans les canaux de communication, émis mais pas encore reçus).
- Configuration (du système) : On parle aussi d'état global. C'est l'état local de tous les sites et l'ensemble de tous les messages en transit.

Exécution

- Sites activables : dans une configuration donnée, les sites activables sont ceux pour lesquels il existe un évènement extérieur qui, après avoir été déclenché, est en attente de traitement.
- Exécution : une séquence de configurations $C_0 C_1 C_2 \dots C_k C_{k+1} \dots$ telle que, entre deux configurations consécutives $C_k C_{k+1}$:
 - tous les sites activables exécutent les actions associées à leur évènement en attente et donc modifient leurs états locaux ;
 - et les autres sites ne changent pas d'état local.

On dit que C_{k+1} est le successeur de C_k

- Un exemple classique = l'exécution d'une règle gardée :
 1. réception d'un message (c'est l'évènement qui était en attente) ;
 2. traitement de ce message (c'est à dire exécution des actions internes liées à la réception du message, et donc changement d'état local du site activé) ;
 3. envoi d'un ou plusieurs message(s).

Le temps de transit fini des messages induit un fort non-déterminisme dans les exécutions d'un algorithme distribué.